UNIVERSITY OF COMPUTER STUDIES (MANDALAY)



SECURE ONLINE VOTING SYSTEM

OBJECT ORIENTED DESIGN AND DEVELOPMENT

(CS-4223)

SUBMITTED BY

GROUP-VIII

FOURTH YEAR (CS-SR)

2025-2026(ACADEMIC YEAR)

Group Member List

No.	RollNo	Name
1.	KPTM_10994	Ma Theint Nadi Soe
2.	KPTM_10995	Ma Hay Mi Kyaw
3.	KPTM_10997	Ma Theint Thinzar Lwin
4.	KPTM-10996	Ma Thet Htar Swe
5.	KPTM-10993	Ma Pyae Theingi Kyaw
6.	KPTM-10998	Ma Chit Su Wai

Supervised By

Daw Theint Theint Htwe

Lecturer

Faculty Of Information Science

University Of Computer Studies (Mandalay)

CONTENTS

Table of Contents	Page
Abstract	
1.Introduction	1
2.Objectives	2
3.Business Process	3
4.Requirement Specification	4
4.1 Functional Requirements	4
4.2 Non-Functional Requirements	5
5. UseCase Digaram	6
Figure5.1 Preliminary Use Case Diagram For Secure Online Voting System	6
6.Text Description	7
Table 6.1 Table: Use Case " Admin Login"	7
Table 6.2 Table: Use Case "Add Voter"	8
Table 6.3 Table: Use Case "Update Voter"	9
Table 6.4 Use Case " Check Voting Status"	10
Table 6.5 Table: Use Case "Display Voterlists"	11
Table 6.6 Table: Use Case "Display Candidatelists"	12
Table 6.7 Table: Use Case "Add Candidate"	13
Table 6.8 Table: Use Case "Update Candidate"	14
Table 6.9 Table: Use Case "Voter Login"	15
Table 6.10 Table: Use Case "View Results"	16
Table 6.11 Table: Use Case "Cast Vote"	17
7.Final UseCase Diagram	18
8.Domain Modeling	19
9.Interaction Modeling	20
9.1 Sequence Diagram	20
9.1.1Sequence diagram for Admin Login	20
9.1.2Sequence diagram for Add Voter	21
9.1.3Sequence diagram for Update Voter	22
9.1.4Sequence diagram for Check Voting Status	23
9.1.5Sequence diagram for Display Voter List	24

9.1.6Sequence diagram for Add Candidate 25
9.1.7Sequence diagram for Update Candidate 26
9.1.8Sequence diagram for Display Candidate List 27
9.1.9Sequence diagram for Voter Login 28
9.1.10Sequence diagram for View Results 29
9.1.11Sequence diagram for Cast Vote 30
10.Class Diagram 31
11.Implementation 32
12.Conclusion 43

Abstract

The Secure Online Voting System is a Windows-based application developed using Java to conduct elections in a secure and simple way. It allows administrators to manage voter details, candidate information, and election results. Voters can log in and cast their vote electronically. The system reduces paperwork, saves time, and improves accuracy. It also prevents duplicate voting and keeps data safe using security features. This application makes the voting process fast, reliable, and easy to use.

1. Introduction

Voting is an important activity in any organization. Traditional voting methods use paper and manual work, which can lead to errors, delays, and security issues. Managing voter records and counting votes manually requires a lot of effort.

The Voting Management System is a Windows-based Java application designed to overcome these problems. It provides a secure platform where voters can cast their votes electronically, and administrators can manage the election process easily. The system stores data safely and ensures that each voter can vote only once. It also helps in generating quick and accurate results.

This application is user-friendly, secure, and efficient. It reduces manual work and improves the overall voting process.

2. Objectives

The objectives of the Voting Management System are:

- To provide a secure electronic voting platform.
- To simplify voter and candidate registration processes.
- To prevent duplicate or unauthorized voting.
- To maintain accurate and tamper-proof election records.
- To allow administrators to easily manage election data and generate reports.
- To provide voters with a simple and user-friendly interface for voting.

3. Business Process

The business process begins when the administrator logs into the system using secure credentials. Once authenticated, the admin registers voters by entering their personal details into the system and also registers the candidates who will participate in the election. Voters then access the system using their assigned login credentials, after which the system verifies whether the voter has already cast a vote. If the voter is eligible, they are allowed to select a candidate and submit their vote. All votes are securely stored in a PostgreSQL database to ensure data integrity and confidentiality. Throughout or after the election period, the administrator can view the election results.

4. Requirement Specification

4.1 Functional Requirements

Admin Module

Admin Login The system shall allow administrators to log in securely. The system shall restrict unauthorized access.

Voter Registration The system shall allow the admin to add, update, and delete voter records. The system shall ensure unique voter identification.

Candidate Registration The system shall allow the admin to register, update, and remove candidates.

View Results The system shall allow the admin to view election results. The system shall display total votes per candidate.

Check Voting Status The system shall allow voters to check their voting status. The system shall confirm successful voting.

User Module

Voter Login The system shall allow voters to log in securely. The system shall validate login credentials.

Cast Vote The system shall allow voters to cast a single vote. The system shall prevent multiple voting.

4.2 Non-Functional Requirements

Security Data shall be encrypted. Unauthorized access shall be prevented.

Performance The system shall support concurrent users. Response time shall be minimal.

Reliability Votes shall be recorded accurately. Data shall be protected from loss.

Usability The system shall be easy to use. The interface shall be user-friendly.

Scalability The system shall support a large number of users.

Availability The system shall be available throughout the election period.

5. UseCase Digaram

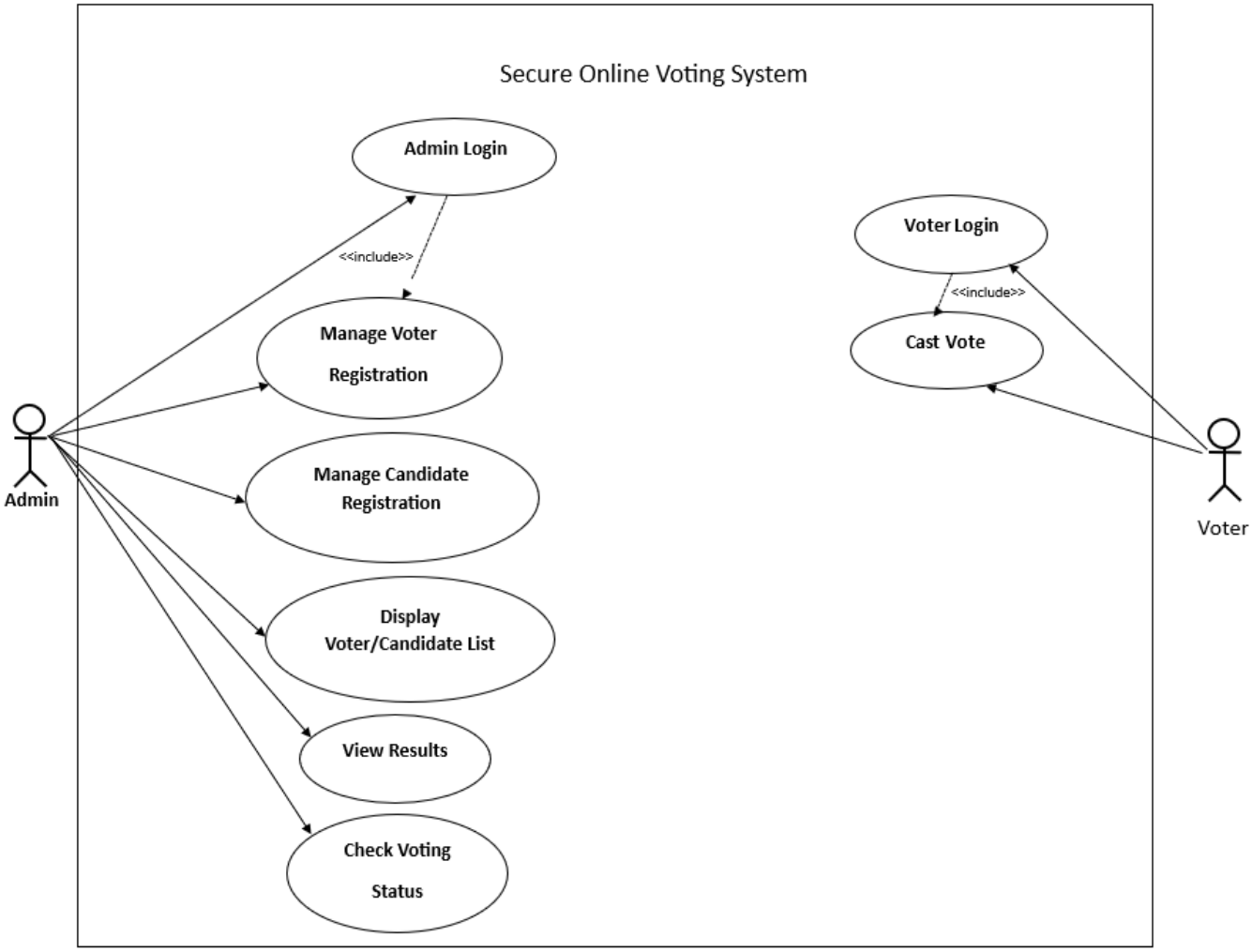


Figure5.1 Preliminary Use Case Diagram For Secure Online Voting System

6 . Text Description

Table 6.1 Table: Use Case " Admin Login"

Actions performed by the actor	Responses from the system
1. Admin clicks the LOGIN button on the User Interface.	
	2. System requests login credentials from the Admin.
3. Admin enters **username and password**.	
	4. User Interface sends login (username, password) request to the Voting System.
	5. Voting System verifies the credentials by sending a query to the Database using executeQuery().
	6. Database processes the query and returns a **ResultSet** to the Voting System.
	7. Voting System returns a boolean result to the User Interface and displays ShowSuccessMessage() if valid or ShowErrorMessage() if invalid.

Table 6.2 Table: Use Case "Add Voter"

Actions performed by the actor	Responses from the system
1.Admin clicks the Add Button on the interface.	
	2.System displays a form and requests voter details.
3.Admin enters voter information (voterId, password, name, etc.).	
	4.System receives the data through AddVoter() from the User Interface.
	5. System processes the data and sends it to the voter object and return voterdata to the system.
	6. The system sends a request to the database to store the voter information.
	7. The database returns the ResultSet to the system.
	8 The system sends a success message to the user interface.
9. The admin views the result on the screen.	
	10.If the admin wants to add another voter, the system goes back to Step 2; otherwise, the process ends.

Table 6.3 Table: Use Case "Update Voter"

Actions performed by the actor	Responses from the system
1. The admin clicks the Update button in the user interface.	
	2. The system requests the voter information to be updated.
3. The admin enters the voter data.	
	4. The user interface sends the request to the voting system using updatevoterlist(voterId).
	5. The voting system sends an update request to the database using executeUpdate(sql).
	6. The database returns the number of rows affected to the system..
	7. The system calls updatevoter(voterId) to update the voter information in the Voter object and return the information to the system.
	8.The system returns updaterowsAffected and executes update(updaterowsAffected) to store changes in the database.
	9.The system confirms success and sends the result to the user interface.
10.The admin views the updated result on the screen.	
	11.If the admin wants to update another voter, the system goes back to Step 2; otherwise, the process ends.

Table 6.4 Use Case " Check Voting Status"

Actions performed by the actor	Responses from the system
1. The admin clicks the Result/Status button in the user interface.	
	2. The user interface sends the request to the system using ActionPerformed().
	3. The system sends a query to the database using executeQuery() to retrieve the vote count.
	4. The database returns the ResultSet (vote count) to the system.
	5. The system processes the result and updates the result set using executeUpdate().
	6. The database confirms Update Success.
	7. The system sends a message to the user interface using showMessageDialog().
8. The user interface displays the voting result/status to the admin	

Table 6.5 Table: Use Case "Display Voterlists"

Actions performed by the actor	Responses from the system
1. The admin selects the Display Voter List option in the user interface.	
	2. The user interface sends a request to the system using userlist() for voter.
	3.The system sends a query to the database using executeQuery("SELECT * FROM addvoter1") for voter.
	4.The database returns the ResultSet to the system.
	5. The system displays the voter data to the user interface.
	6.The user interface displays the voter list to the admin.
7.The admin views voterlists.	

Table 6.6 Table: Use Case "Display Candidatelists"

Actions performed by the actor	Responses from the system
1. The admin selects the Display Candidate List option in the user interface.	
	2. The user interface sends a request to the system using userlist2() for candidate .
	3.The system sends a query to the database using executeQuery("SELECT * FROM partylist1") for candidate.
	4.The database returns the ResultSet to the system.
	5. The system displays the candidate data to the user interface.
	6.The user interface displays the candidate list to the admin.
7.The admin views candidatelists.	

Table 6.7 Table: Use Case "Add Candidate"

Actions performed by the actor	Responses from the system
1.Admin clicks the Add Button on the interface.	
	2.System displays a form and requests candidate details.
3.Admin enters candidate information (name, age, etc.).	
	4.System receives the data through AddCandidate () from the User Interface.
	5. System processes the data and sends it to the candidate object and return candidate data to the system.
	6. The system sends a request to the database to store the candidate information.
	7. The database returns the ResultSet to the system.
	8 The system sends a success message to the user interface.
9. The admin views the result on the screen.	
	10.If the admin wants to add another candidate, the system goes back to Step 2; otherwise, the process ends.

Table 6.8 Table: Use Case "Update Candidate"

Actions performed by the actor	Responses from the system
1. The admin clicks the Update button in the user interface.	
	2. The system requests the candidate information to be updated.
3. The admin enters the candidate data.	
	4. The user interface sends the request to the voting system using updatecandidatelist(name).
	5. The voting system sends an update request to the database using executeUpdate(sql).
	6. The database returns the number of rows affected to the system..
	7. The system calls updatecandidate(name) to update the voter information in the Candidate object and return the information to the system.
	8.The system returns updaterowsAffected and executes update(updaterowsAffected) to store changes in the database.
	9.The system confirms success and sends the result to the user interface.
10.The admin views the updated result on the screen.	
	11.If the admin wants to update another candidate, the system goes back to Step 2; otherwise, the process ends.

Table 6.9 Table: Use Case "Voter Login"

Actions performed by the actor	Responses from the system
1. The voter clicks the LOGIN button on the user interface.	
	2.The system requests the voter to enter login information.
3. The voter enters voter ID and password.	
	4. The user interface sends the login request to the voting system using login(voterId, password)
	5. The system sends a query to the database using executeQuery() to verify the voter information.
	6. The database returns the ResultSet to the system.
	7. The system checks the result and returns a boolean value to the user interface.
	8. If the login is valid, the system displays ShowSuccessMessage().
	9. If the login is invalid, the system displays ShowErrorMessage()

Table6.10 Table: Use Case "View Results"

Actions performed by the actor	Responses from the system
1.The admin clicks the button to view the winner result.	
	2. The User Interface sends showuser() request to the Voting System.
	3. The Voting System sends executeQuery() request to the Database.
	4. The Database returns the Resultset to the Voting System.
	5. The Voting System sends the winner result to the User Interface.
6. The admin views the winner result.	

Table 6.11 Table: Use Case "Cast Vote"

Actions performed by the actor	Responses from the system
1. Voter clicks the vote button on the User Interface.	
	2.User Interface captures the action and sends ActionPerformed() to the Voting System.
	3.Voting System sends executeQuery("SELECT vote_count…") to the Database.
	4. Database returns the current vote count to the Voting System.
	5.Voting System updates the vote result in the Database using executeUpdate().
	6. Database confirms the update to the system.
	7.The system confirm showmessage dialog to the user interface.
	8. The User Interface displays a confirmation message to the Voter.

7. Final UseCase Diagram

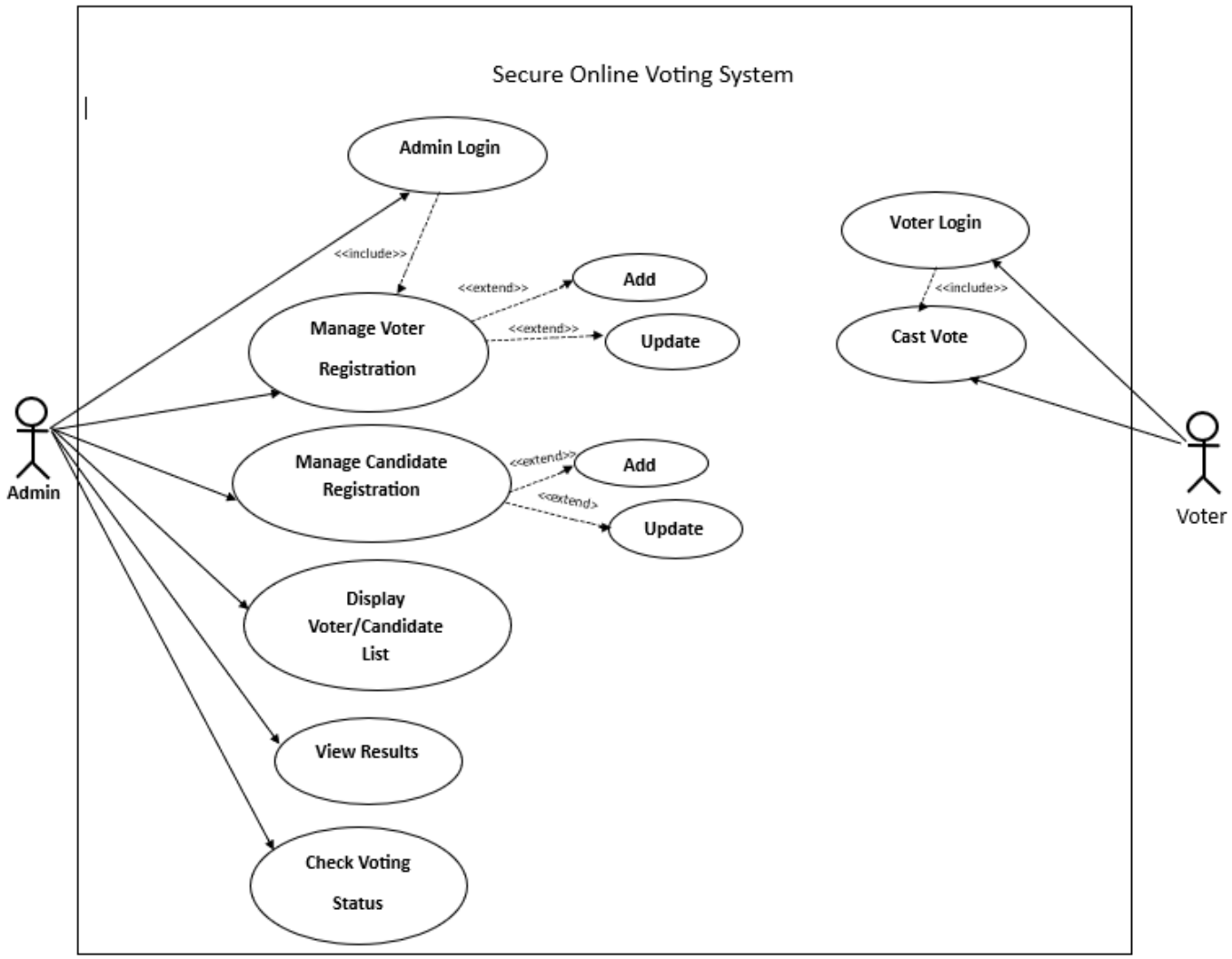


Figure: Final UseCase Diagram

8.Domain Modeling

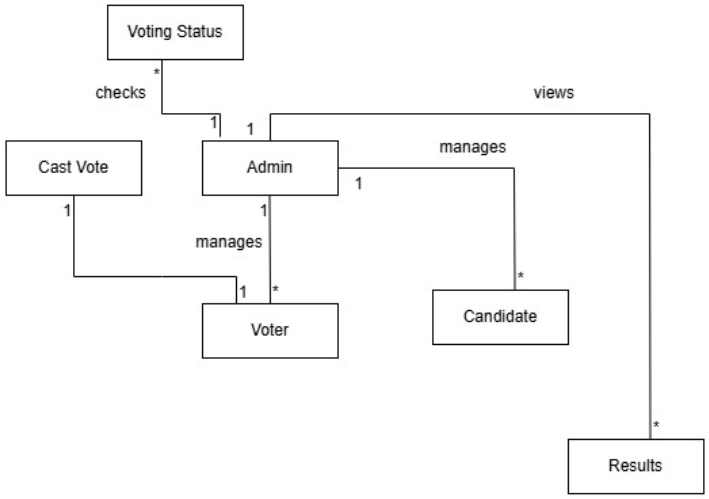


Figure:Conceptual Model including classes and their associations For Secure Online Voting System

9. Interaction Modeling

9.1. Sequence Diagram

9.1.1 Admin Login

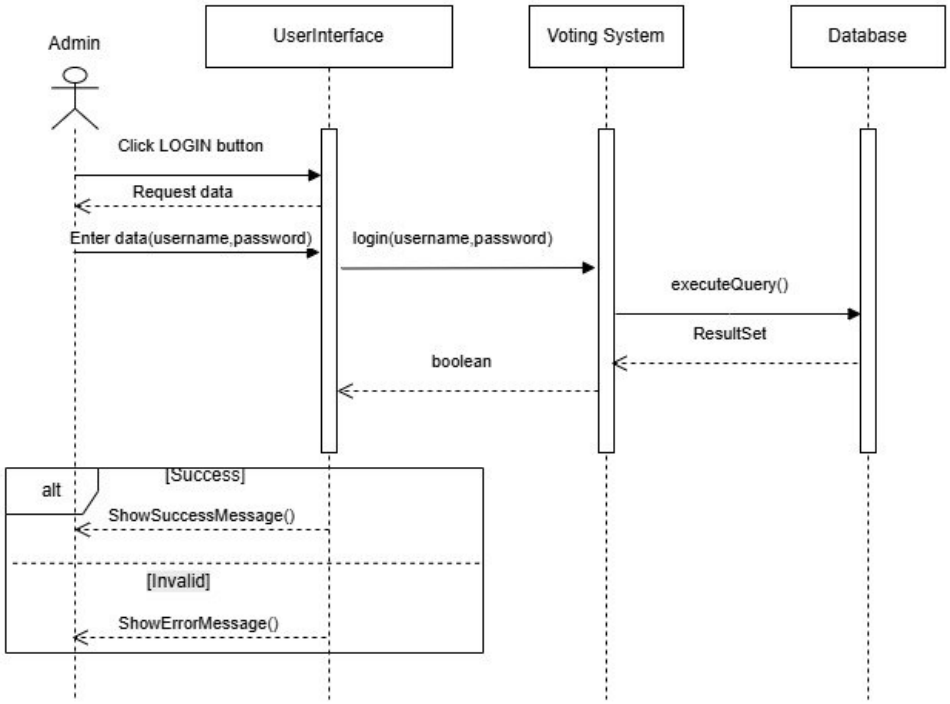


Figure 9.1.1 Sequence Diagram for Admin Login

9.1.2 Add Voter

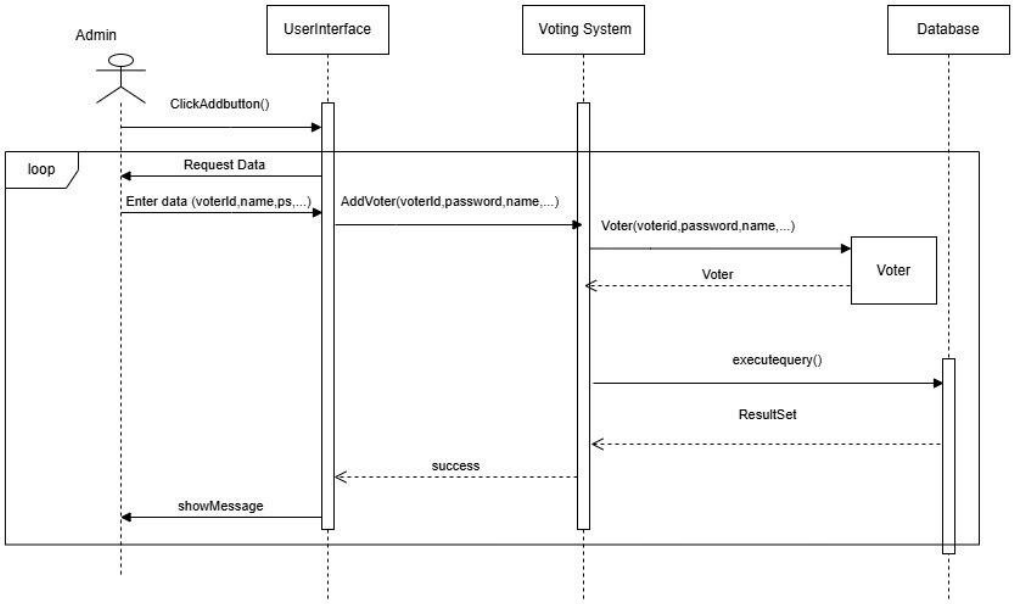


Figure 9.1.2 Sequence Diagram for Add Voter

9.1.3 Update Voter

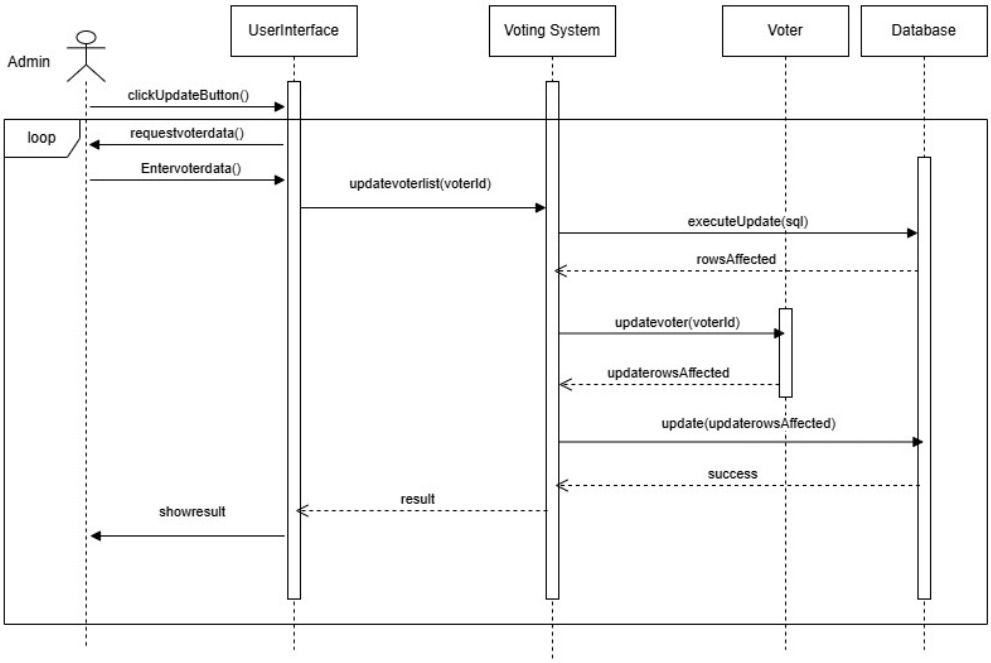


Figure 9.1.3 Sequence Diagram for Update Voter

9.1.4 Check Voting Status

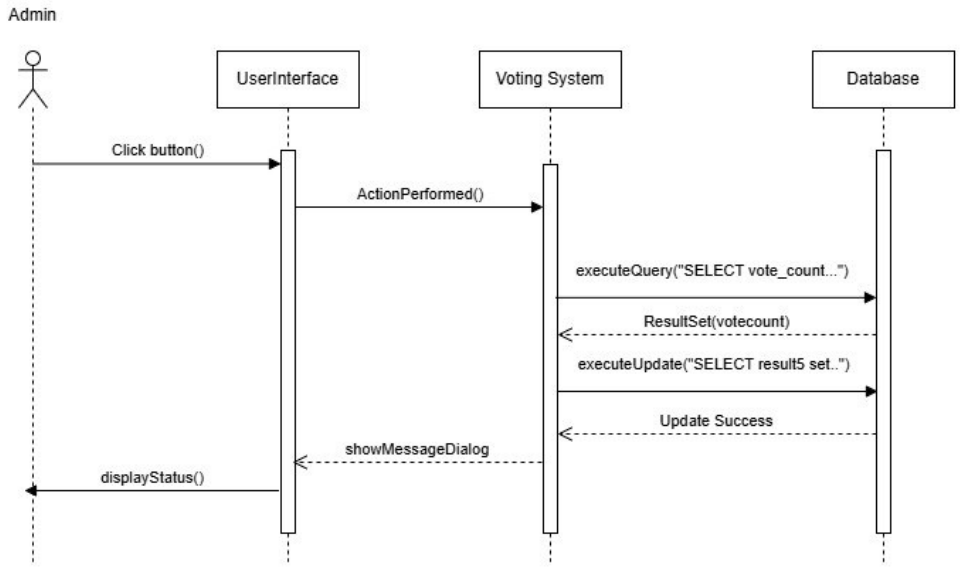


Figure 9.1.3 Sequence Diagram for Check Voting Status

9.1.5 Display Voter List

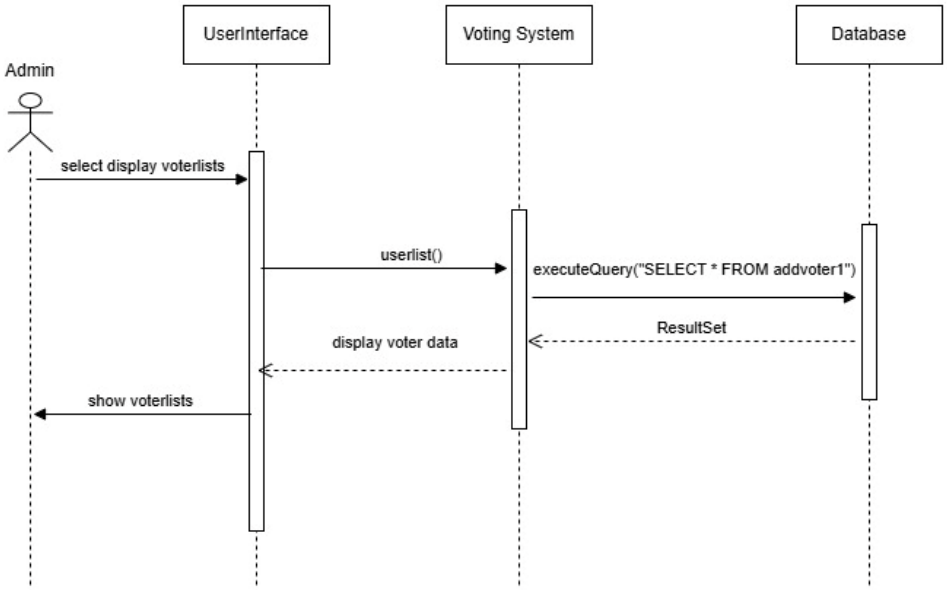


Figure 9.1.5 Sequence Diagram for Display Voter List

9.1.6 Add Candidate

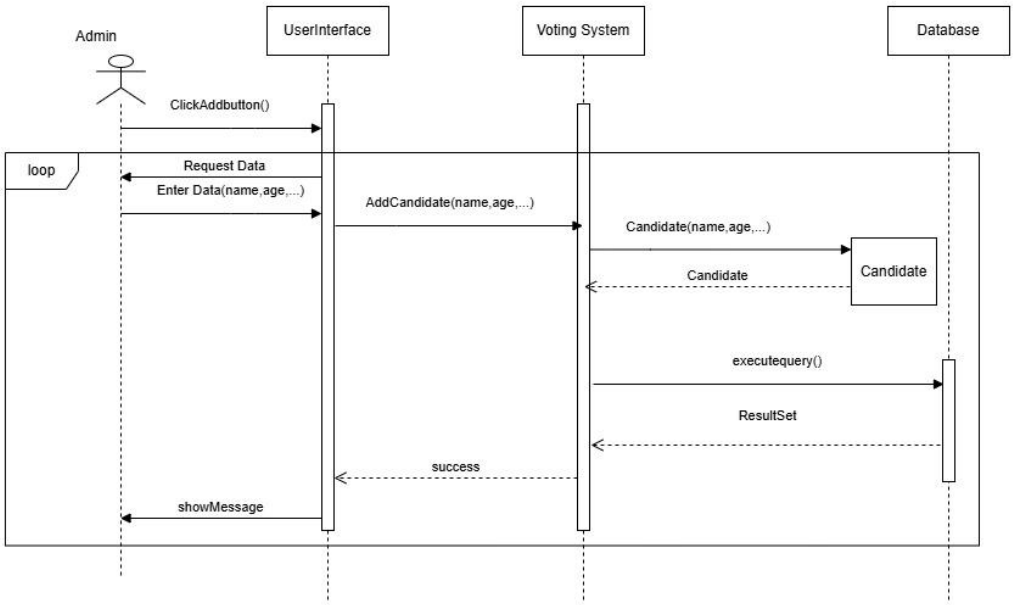


Figure 9.1.6 Sequence Diagram for Add Candidate

9.1.7 Update Candidate

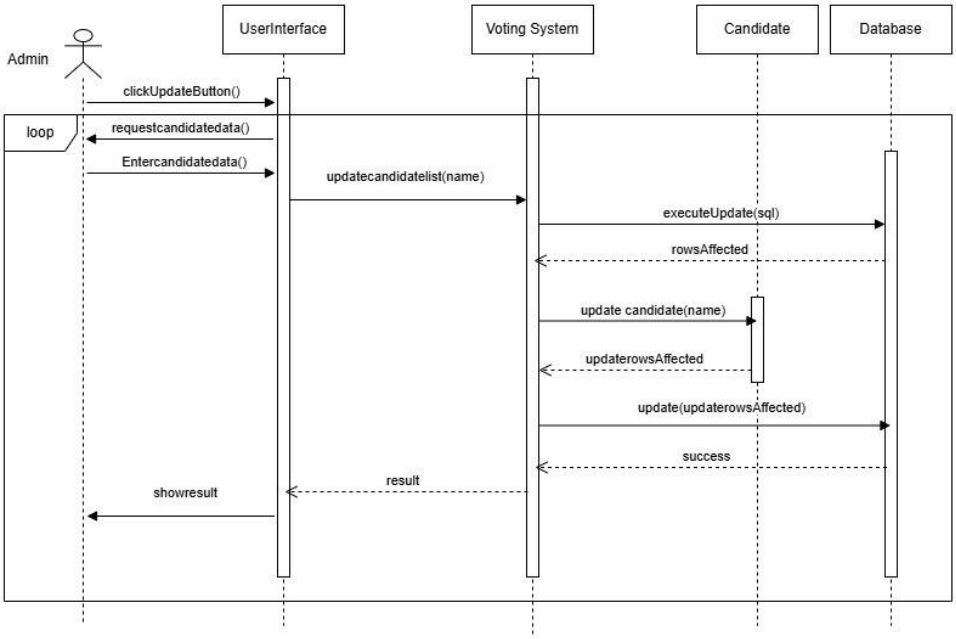


Figure 9.1.7 Sequence diagram for Update Candidate

9.1.8 Display Candidate List

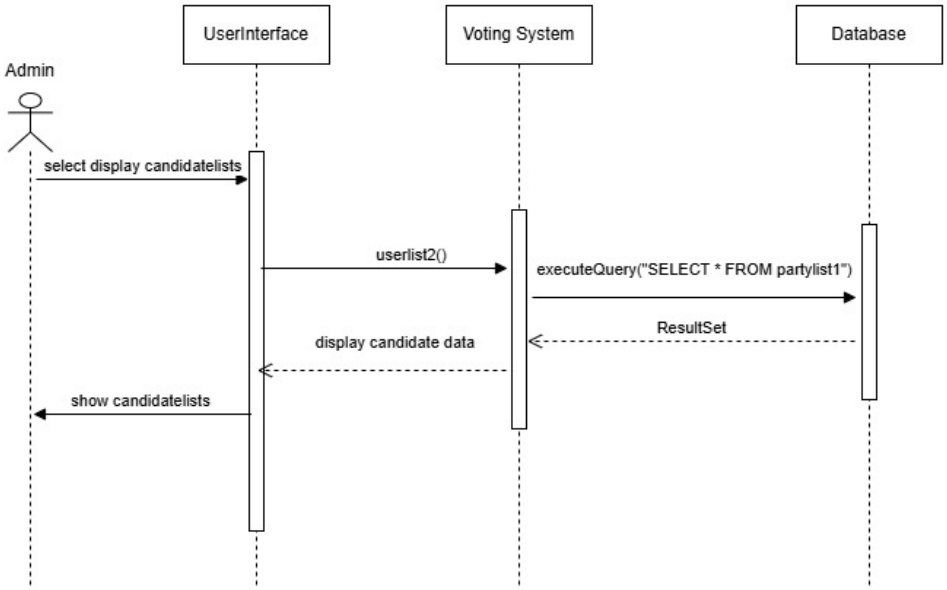


Figure 9.1.8 Sequence diagram for Display Candidate List

9.1.9 Voter Login

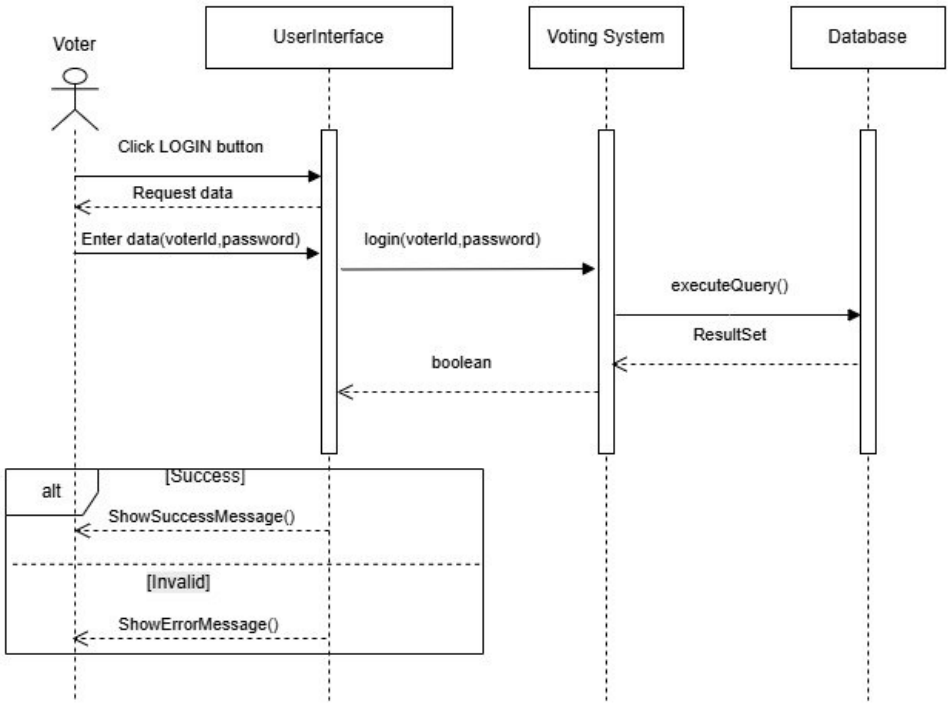


Figure 9.1.9 Sequence diagram for Voter Login

9.1.10 View Results

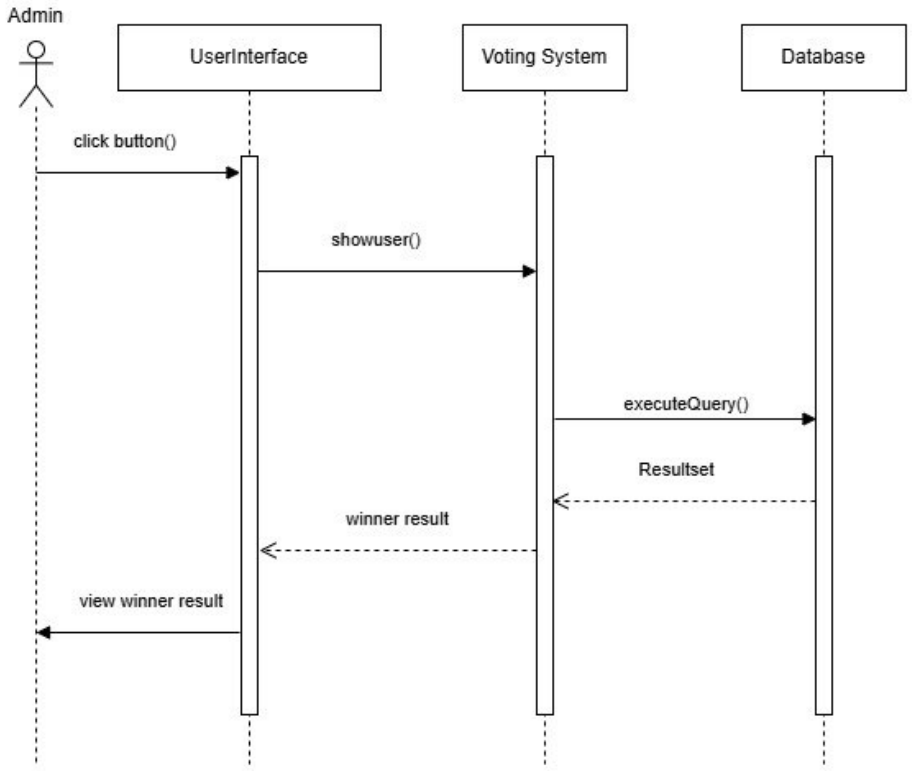


Figure 9.1.10 Sequence diagram for View Results

9.1.11 Cast Vote

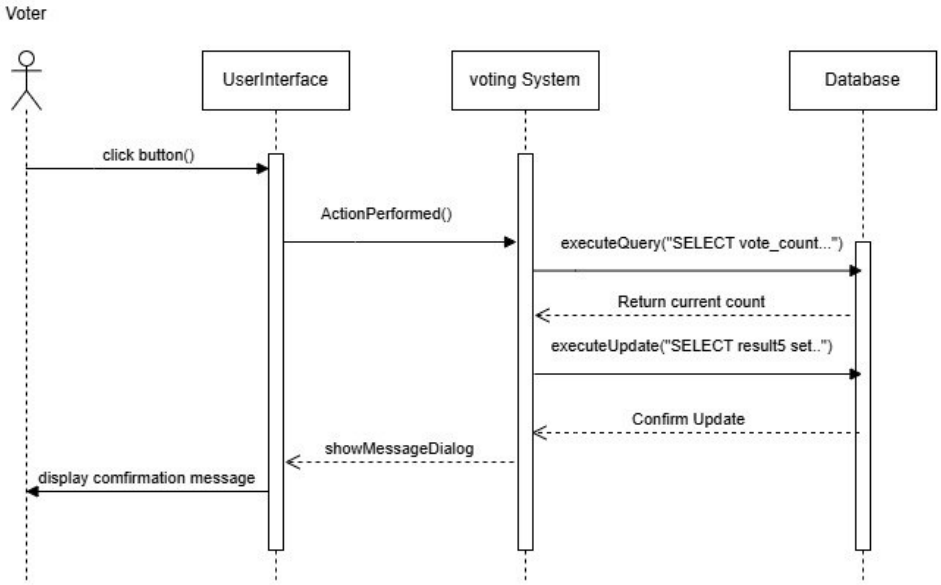


Figure 9.1.11 Sequence diagram for Cast Vote

10.Class Diagram

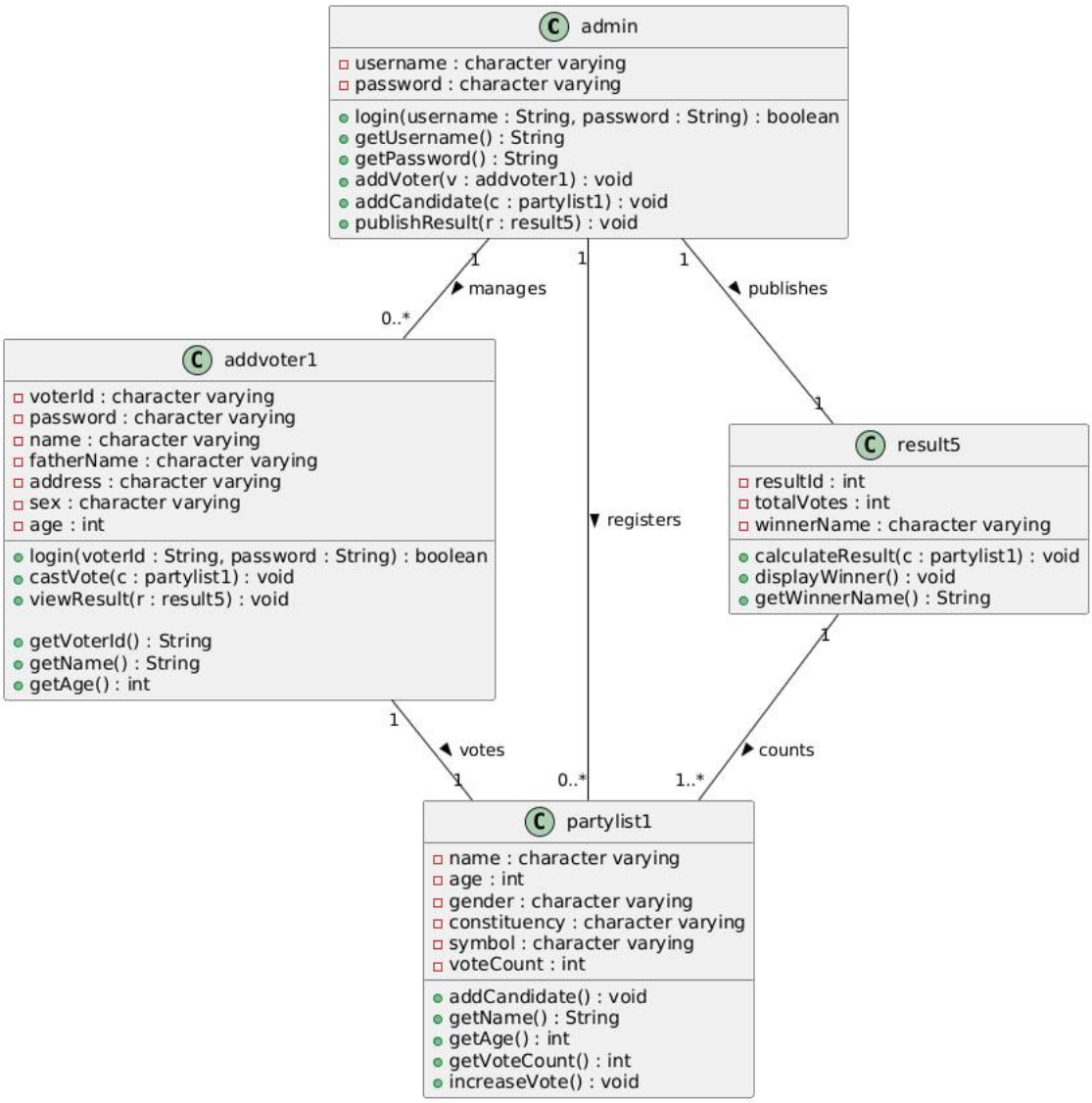
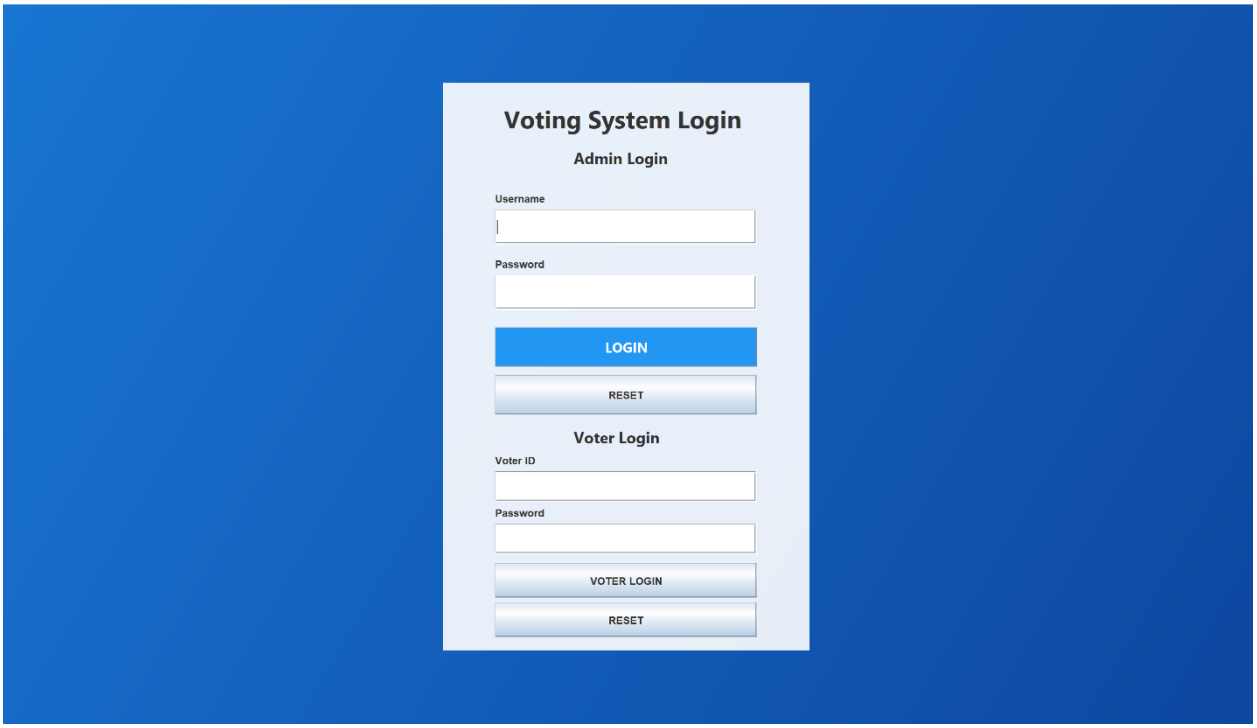


Figure: Class Diagram for Secure Online Voting System

11.Implementation

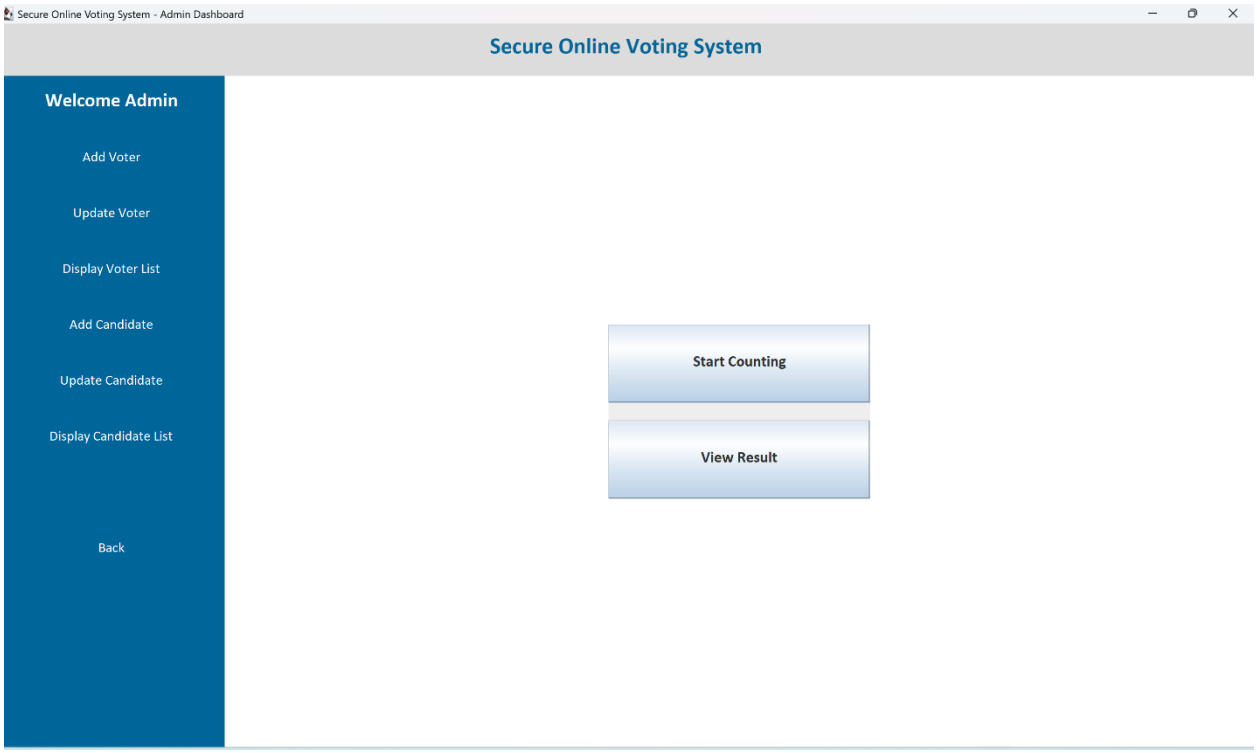
11.1 Login

This page implements the login module of the Voting Management System, providing separate authentication interfaces for Admin and Voter users. The Admin login allows authorized administrators to access system management functions using a username and password, while the Voter login verifies voter credentials using voter ID and password before allowing access to the voting process. Input validation and reset options are included to ensure secure and user-friendly authentication.



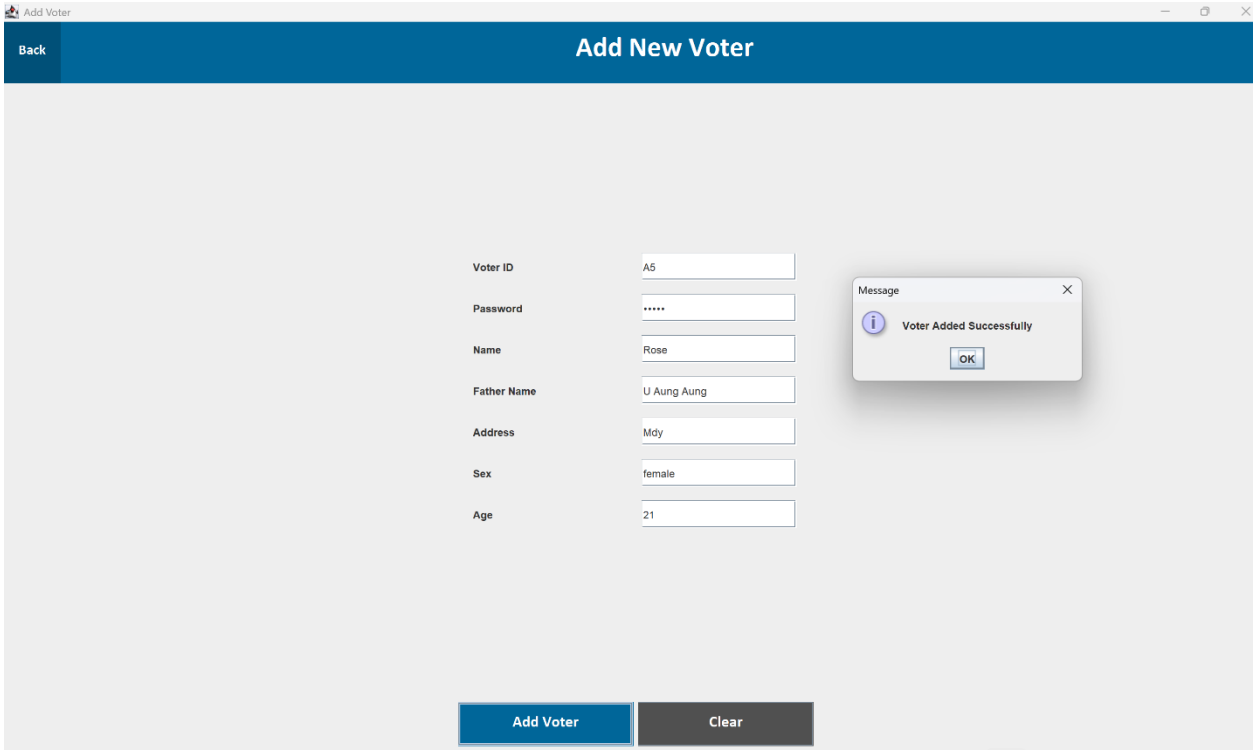
11.2 Admin home page

In the home page there are six options that only Admin can view add voter, update voter, display voter list,add candidate,update candidate,display candidate list,counting and result.



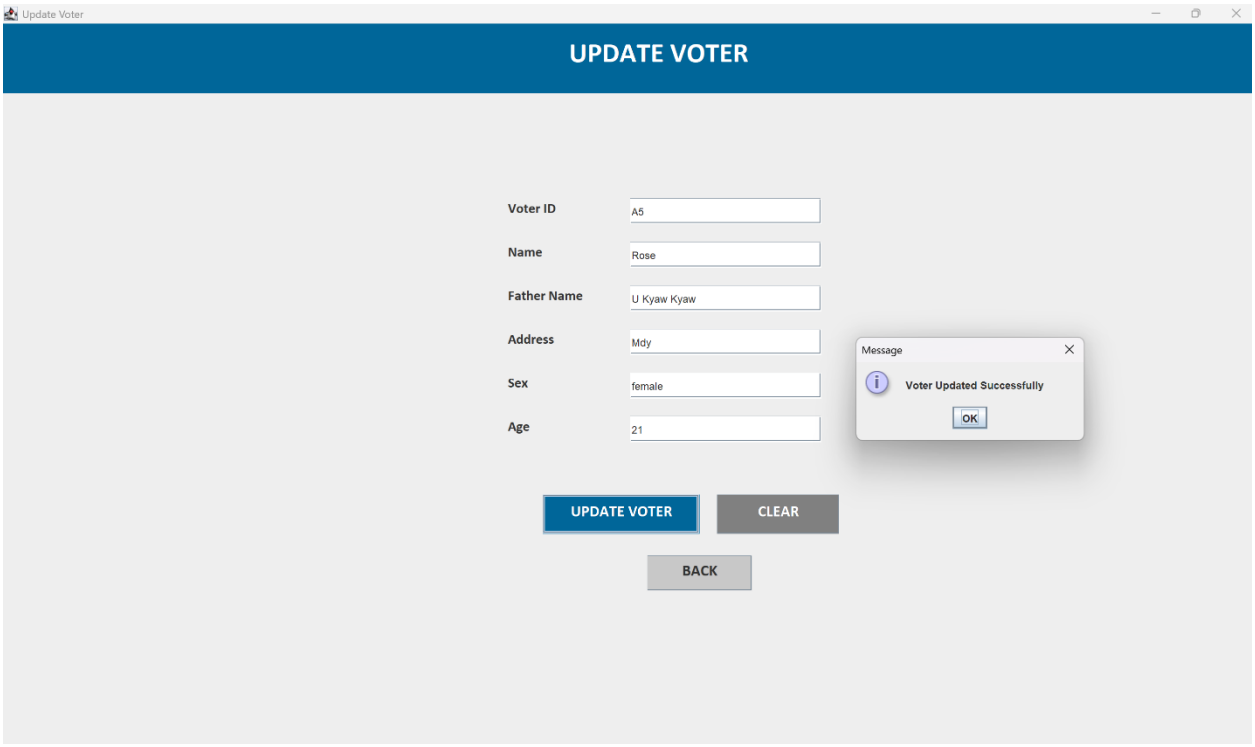
11.3 Add voter

This page is used by the admin to add new voter details into the system.When the admin clicks “Add Voter” from the dashboard menu, they arrive at the Add Voter Page. This page shows a form with fields for Voter ID, Password, Father Name, Address, Sex, and Age. The admin fills in the details and clicks the “Add Voter” button. The system then saves the voter information in the database and shows a confirmation message. If any required field is missing or the Voter ID already exists, an error message appears.



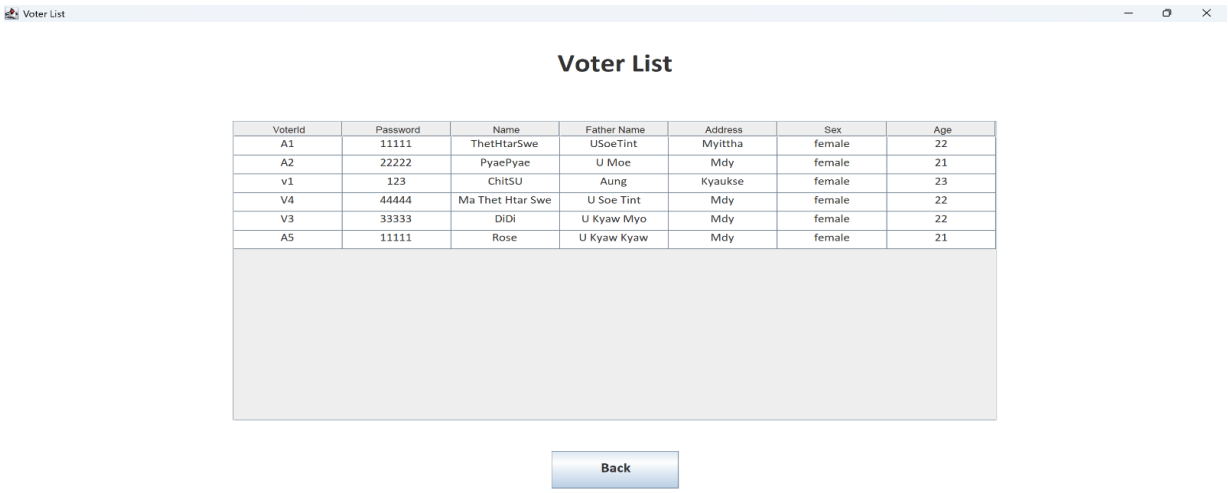
11.4 Update Voter

This page is used by the admin to update voter details. When the admin selects “Update Voter” from the dashboard, they see a page with a form to search and edit a voter. The admin can enter the Voter ID to find the voter’s existing details. The system displays the current Password, Father Name, Address, Sex, and Age in editable fields. After making changes, the admin clicks “Update Voter”, and the system saves the updated information in the database and shows a confirmation message. If the Voter ID is not found or any field is invalid, an error message appears.



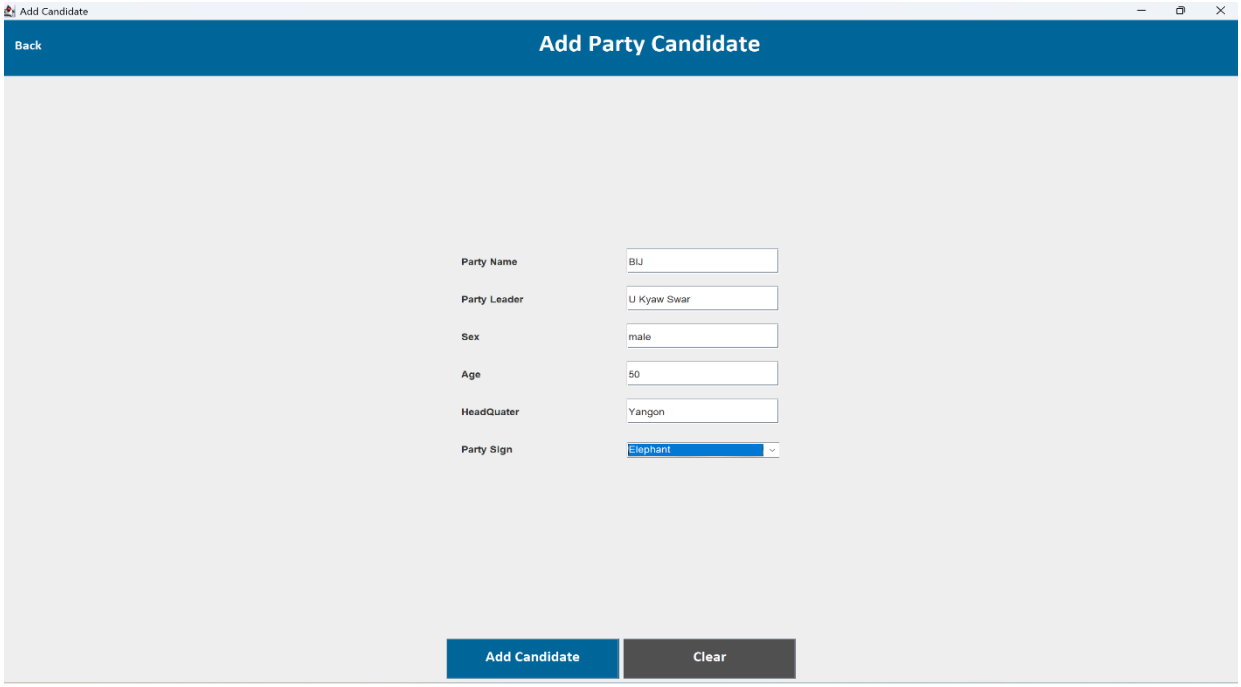
11.5 Display Voter

When the admin clicks “Display Voter” from the dashboard, the system shows a list of all registered voters in a table. Each row displays the Voter ID, Father Name, Address, Sex, and Age. The admin can scroll through the list to view voter details.



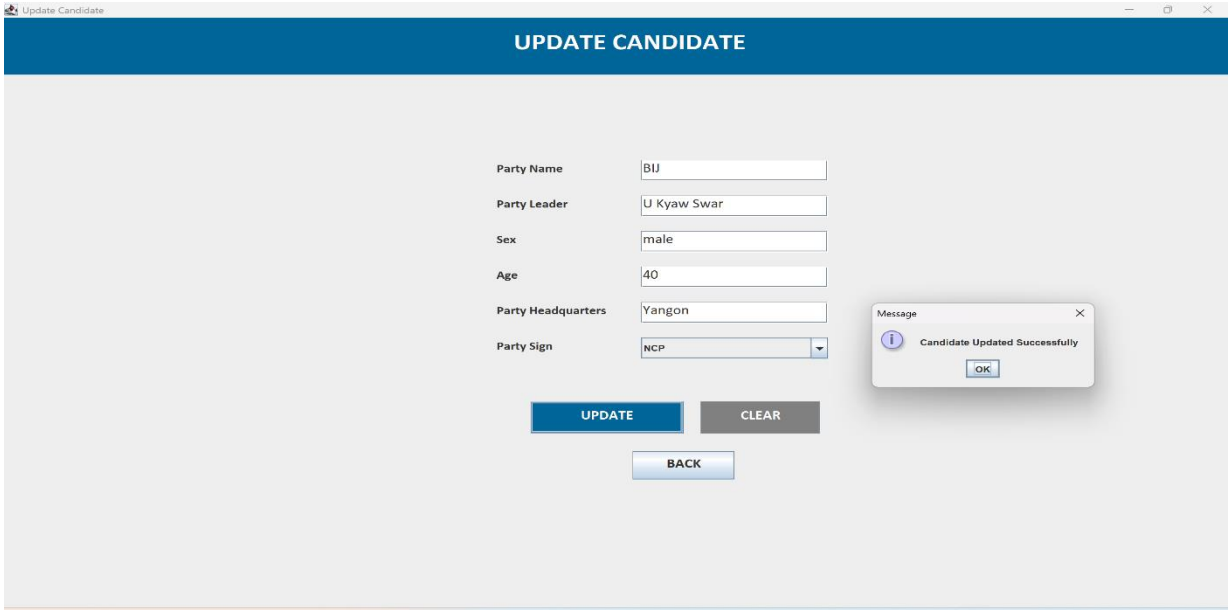
11.6 Add Candidate

This page is used by the admin to add new Candidate details into the system.When the admin clicks “Add Candidate” from the dashboard, they arrive at the Add Candidate Page. This page shows a form with fields for Party Name, Party Leader, Sex, Age, Headquarter, and Party Sign. The admin fills in the details and clicks the “Add Candidate” button. The system saves the candidate in the database and shows a confirmation message.



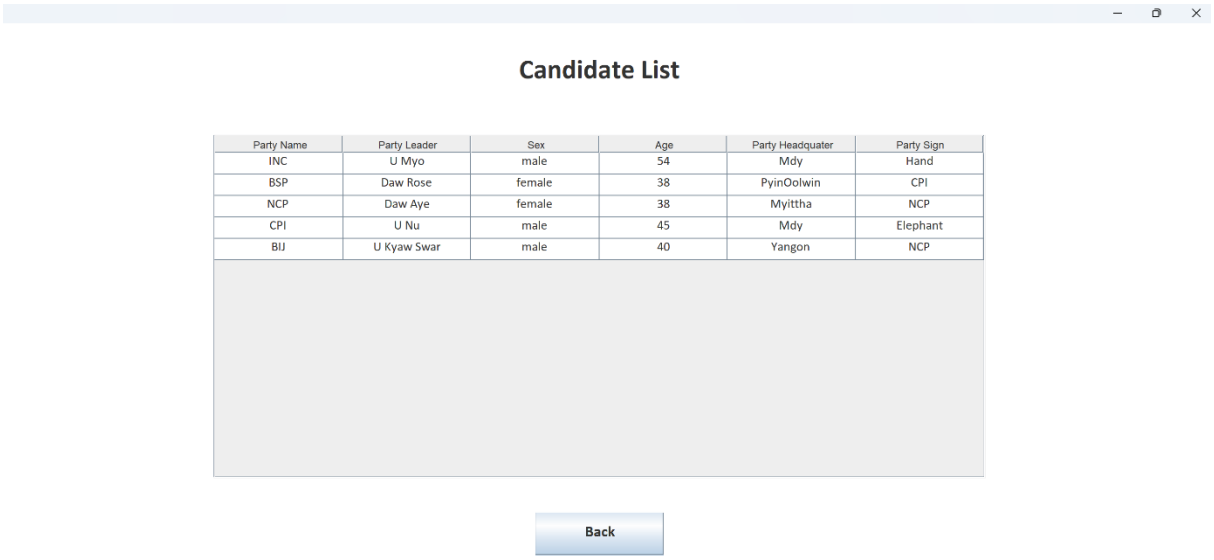
11.7Update Candidate

This page is used by the admin to update candidate details. When the admin clicks “Update Candidate” from the dashboard, they see a page with a form to search and edit a candidate. The admin enters the Party Name or to load the candidate’s current details. They can then edit Party Leader, Sex, Age, Headquarter, and Party Sign and click “Update Candidate”. The system saves the changes in the database and shows a confirmation message.



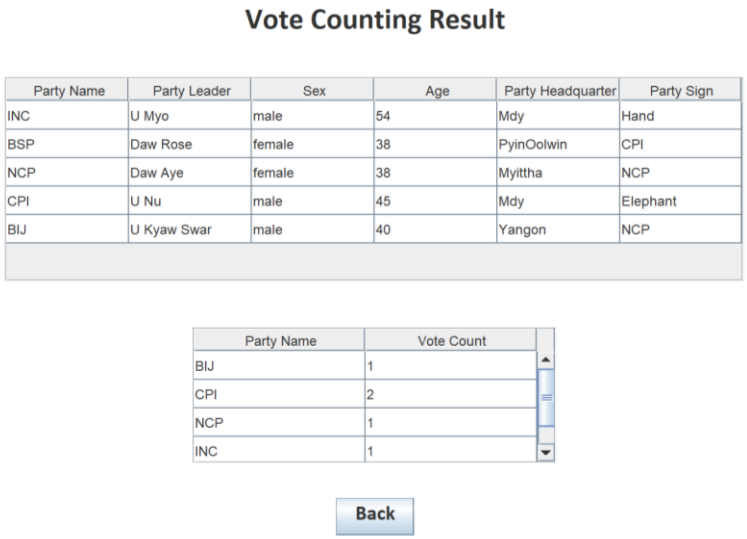
11.8 Display Candidate List

When the admin clicks “Display Candidate” from the dashboard, the system shows a list of all registered candidates in a table. Each row displays Party Name, Party Leader, Sex, Age, Headquarter, and Party Sign. The admin can scroll through the list to view candidate details.



11.9 Start Counting

When the admin clicks “Start Counting”, the system shows the Candidate List Table with all candidates and a Result Table showing each party’s name and the number of votes it received. The system automatically counts votes from the voters and updates the result table, showing zero if no votes have been cast.



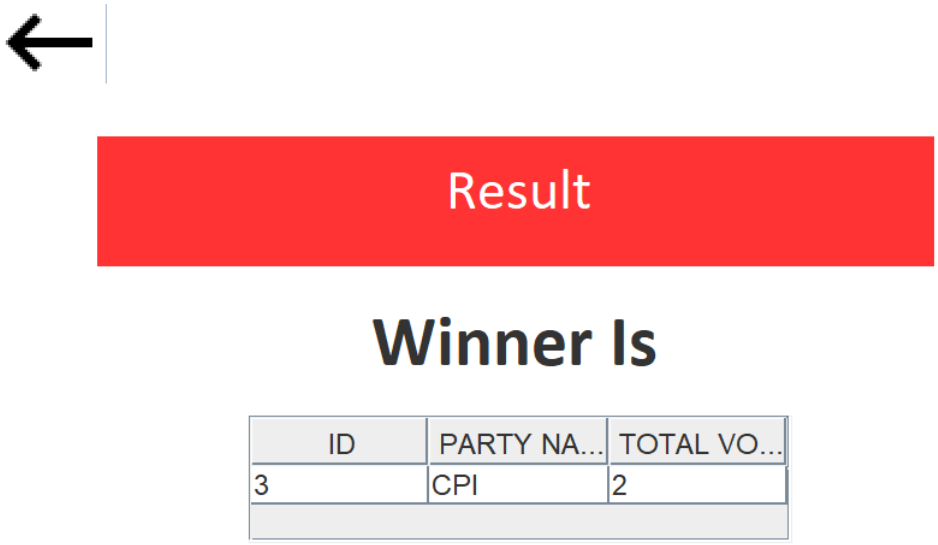
11.10 Voter home page

This page allows voters to start the election and select a candidate to cast their vote.



11.11 View Result

When the admin clicks “Start Counting”, the system shows the Candidate List Table with all candidates and a Result Table showing each party’s name and the number of votes it received. The system automatically counts votes from the voters and updates the result table, showing zero if no votes have been cast.



11.Conclusion

The **Secure Voting Management System** provides a reliable and efficient solution for conducting elections electronically. By replacing traditional paper-based voting methods, the system reduces manual work, minimizes errors, and improves the overall speed and accuracy of the election process.

The system allows administrators to easily manage voter and candidate information, while voters can securely log in and cast their votes through a simple and user-friendly interface. Security features ensure that each voter can vote only once and that all voting data is stored safely in the database. This helps maintain transparency, data integrity, and trust in the election process.

Overall, the Secure Voting Management System demonstrates how technology can simplify election management while ensuring security and accuracy. It provides a practical and effective approach for organizations or institutions that require a secure and automated voting solution. Future improvements can include enhanced security features, web or mobile support, and integration with advanced authentication methods to further improve the system.